TRIPLETEN QA ENGINEER PROGRAM · FINAL PROJECT

Full-Stack Quality Assurance: Urban Scooter

Manual testing across web, mobile and API, with defects tracked in Jira

Michael Andeck

mikeandeck@gmail.com · linkedin.com/in/mandeck · github.com/mandeck/Portfolio

109	77 / 29 / 3	22	3
test cases designed and executed	passed / failed / skipped	distinct bugs logged	areas: web, mobile, API

Key findings

- **Web:** 11 of 65 cases failed, giving 9 bugs. The most significant: the Last name field accepts 16+ characters (S9P-2), and short or empty Address entries give no error (S9P-5).
- **Mobile:** the two-hour order notification never appears (S9P-10) and prevented three further test cases. Offline behavior produced 11 bugs, including “Order accepted” flashing with no connection (S9P-17).
- **API:** 8 of 9 cases passed. Deleting a courier with no ID returns error 404 instead of error 400 (S9P-22).

Contents

PART	AREA	WHAT WAS TESTED	CASES	BUGS	PAGE
1	**Web testing**	Order form in Chrome and Opera, equivalence partitioning and boundary values	65	9	2
2	**Mobile testing**	Android app in an emulator: notifications and no-internet behavior, Figma comparison	35	12	6
3	**API testing**	Courier add and delete endpoints in Postman	9	1	10
	Summary	Combined results and automation potential			13

A training project on a practice application, completed as part of coursework. Tools: Google Sheets, Jira, Postman, Android Studio emulator, Figma, Chrome and Opera. S9P numbers are reference IDs from the project’s Jira board.

FULL-STACK MANUAL TESTING · PART 1 OF 3

Web Testing: Customer Order Form

Urban Scooter sample web application.

65	2	54 / 11	9
test cases designed	browsers: Chrome and Opera, 1280×720	passed / failed (same in both browsers)	distinct bugs logged

The application

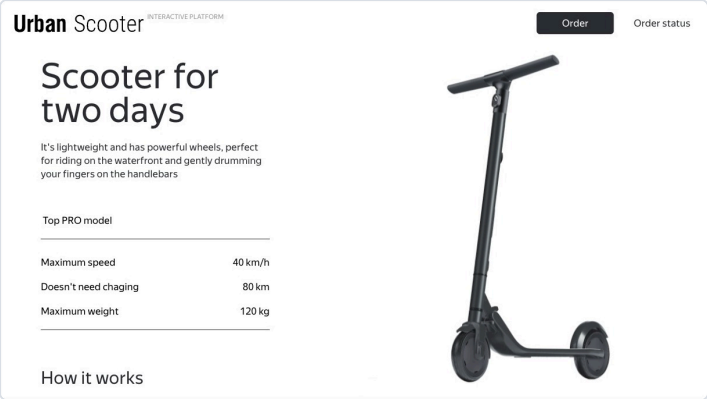


Figure 1. Urban Scooter home page (Chrome, 1280×720).

Urban Scooter is a practice scooter-rental service built for this course. A customer enters their details to order a scooter, and a courier delivers it using the mobile app. The final project tests the site's order form, the courier's Android app and the back-end API (the three parts of this project).

Scope

The assignment was to design positive and negative test cases for the **Customer Details** form, headed "Who the scooter is for." This is the first stage of the Place Order flow. I ran each case in both browsers, recorded results in a Google Sheets workbook, and logged defects in Jira with a temporary link in the matching row.

The form has five required fields: **First name**, **Last name**, **Address**, **Subway station** and **Phone**. The written requirements were the authoritative source. Figma layouts served as a visual reference.

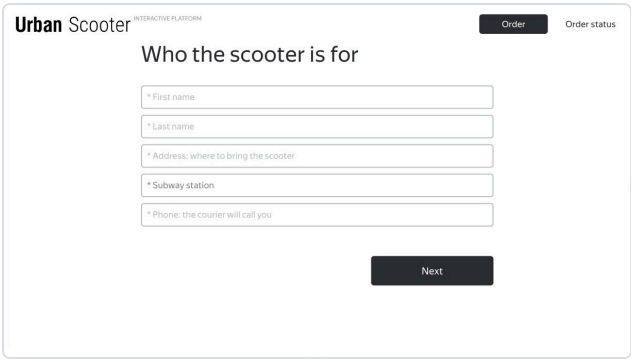


Figure 2. The Customer Details form on load (Chrome, 1280×720).

Environment and tools

Computer	iMac, macOS Sonoma 14.6.1 (23G93)
Browsers	Google Chrome 154.0.8037.58 (arm64) and Opera One 136.0.6008.52 (arm64), both at 1280×720
Tools	Google Sheets (test cases and results), Jira (defects), Postman (cases 62–63, subway list data), Chrome DevTools (responsive 1280×720 view)
Techniques	Equivalence partitioning, boundary value analysis, positive and negative testing, cross-browser execution

Test design

I turned the field rules from the requirements into equivalence classes and boundary values, then wrote one test case per class or rule. Each case records its precondition, steps, class, boundary data, expected result, and the actual result in each browser.

Requirements and coverage by field

FIELD	REQUIREMENT UNDER TEST	CASES	CLASSES AND BOUNDARIES COVERED
First name	Latin letters, spaces, dashes; 2–15 characters. Invalid: red highlight and “Enter correct name.”	14	Lengths 2–15 (boundaries 2, 3, 14, 15), 0–1, and 16–17. Accepted: letters, spaces, dashes. Rejected: periods, commas, numbers, special and non-Latin characters, empty. Highlight and error text.
Last name	Same character and length rules. Error text: “Enter a valid last name”	14	Same structure as First name.
Address	Latin letters, numbers, spaces, dashes, dots, commas; 5–50 characters; leading and trailing spaces trimmed on blur. Error text: “Enter a valid address”	16	Lengths 5–50 (boundaries 5, 6, 49, 50), shorter, and 51–52. Accepted: letters, numbers, spaces, dashes, periods, commas. Rejected: special and non-Latin characters, empty. Space trimming. Highlight and error text.
Phone	Numbers and “+” only; plus sign required; 10–12 characters. Error text: “Enter a valid phone number.”	16	Lengths 10–12 (boundaries 10, 11, 12), 0–9, and 13–14. Plus sign required. Rejected: letters, spaces, dashes, periods, commas, special and non-Latin characters, empty. Highlight and error text.
Subway station	Los Angeles stations only; list comes from the back-end API; required.	5	Outside-LA station not listed; list compared with the API response (Postman, cases 62–63); a station can be selected; cannot advance without one.
Total		**65**	

	A	B	C	D	E	F	G	H	I	J	K
1	ID	Test Case	Precondition	Test Steps	Equivalence Class	Boundary Values	Test Data for Equivalence Class	Test Data for Boundary Values	Expected Result	Actual Result in Chrome 1280x720	Actual Result in Opera 1280x720
13	12	Verify that empty fields are not allowed in the First Name field	1. Server is started 2. "Order" has been clicked	Leave the First Name field empty	Empty field		" " (0)		Empty field is not accepted in the First Name field	PASS	PASS
14	13	Verify that the First Name field is highlighted in red when invalid data is entered	1. Server is started 2. "Order" has been clicked	Type invalid data in the First Name field	Invalid data entered		Alex Jr 5		The First Name field is highlighted in red	PASS	PASS
15	14	Verify that the error text "Enter correct name" is displayed when invalid data is entered	1. Server is started 2. "Order" has been clicked	Type invalid data in the First Name field	Invalid data entered		Alex Jr 5		The error text "Enter correct name" is displayed	FAIL	FAIL
16	15	Verify that only 2-15 characters are allowed in the Last Name field	1. Server is started 2. "Order" has been clicked	Type 2-15 characters in the Last Name field	2-15 characters	2, 3, 14, 15	Alexis (6)	Al(2) Ale(3) Jonathon-Mikey (14) Jonathon-Michel (15)	2-15 characters are accepted in the Last Name field	PASS	PASS
17	16	Verify that less than 2 characters are not allowed in the Last Name field	1. Server is started 2. "Order" has been clicked	Type 0, 1 characters in the Last Name field	0,1 characters	0, 1	"" "A"	"" (0) "A" (1)	0, 1 characters are not accepted in the Last Name field	PASS	PASS
18	17	Verify that more than 15 characters are not allowed in the Last Name field	1. Server is started 2. "Order" has been clicked	Type 16,17 characters in the Last Name field	16,17 characters	16, 17	Jonathon-Michaele (17)	Jonathon-Michael (16) Jonathon-Michaele (17)	16+ characters are not accepted in the Last Name field	FAIL	FAIL
19	18	Verify that Latin letters are allowed in the Last Name field	1. Server is started 2. "Order" has been clicked	Type 4-13 characters (non-BV, in range), Latin letters in Last Name field	Latin letters		Alex		Latin letters are accepted in the Last Name field	PASS	PASS

Figure 3. Excerpt of the test case sheet (IDs 12–18). Each row holds its precondition, steps, equivalence class, boundary data, expected result, and per-browser results. Case 17 (highlighted) failed and links to a bug, which is shown in full in the featured defect.

Results

All 65 cases ran in both Chrome and Opera. The two browsers gave the same result on every case, so no browser-specific defects appeared. That is 130 case executions in total.

FIELD	CASES	PASSED	FAILED		BUGS
First name	14	13	1		1
Last name	14	12	2		2
Address	16	12	4		3
Phone	16	12	4		3
Subway station	5	5	0		0
Total	**65**	**54**	**11**		**9**

Counts are per browser. Green = passed, red = failed.

> **Failed cases are not the same as defects.** The 11 failed cases trace to 9 distinct bugs, because two bugs (S9P-5 and S9P-7) each explain two failed cases. I counted distinct bugs as the number of defects found.

Defects logged

JIRA	FIELD	CASE	EXPECTED (FROM TEST CASE)	ACTUAL	SEVERITY
S9P-1	First name	14	Error text “Enter correct name”	Displays “Enter a valid name” instead	Low
S9P-2	Last name	17	16+ characters rejected	16+ characters are allowed	Medium
S9P-3	Last name	28	Error text “Enter a valid last name”	Displays “Enter a valid name”	Low
S9P-4	Address	29	5–50 characters accepted	50-character addresses are not allowed	Low
S9P-5	Address	30, 40	Fewer than 5 characters, and an empty field, not accepted	No error is indicated and no error message is produced	Medium
S9P-6	Address	44	Error text “Enter a valid address”	Displays “Enter a valid number”	Low
S9P-7	Phone	45, 47	10–12 characters accepted; 13+ rejected	11–13 characters are accepted (10 is rejected, 13 is accepted)	Low–Medium
S9P-8	Phone	50	“+” symbol required	Entry without the “+” symbol is accepted	Low–Medium
S9P-9	Phone	60	Error text “Enter a valid phone number.”	Displays “Enter a valid number.”	Low

Four bugs (S9P-1, 3, 6 and 9) concern the wording of an error message, and S9P-5 is a missing error message. The rest concern length limits or required-input rules. S9P numbers are reference IDs from the project’s Jira board.

Severity scale. Low: cosmetic, or a small departure from the written requirements. Medium: a potentially functional problem (such as a missing limit or feedback) with possible negative effects, but not catastrophic. High: a core feature fails, or a serious risk for this kind of app (such as security or data protection). Ranges mark borderline cases.

Featured defect: S9P-2

Last Name field accepts more than 15 characters (test case 17, Chrome and Opera). Medium severity.

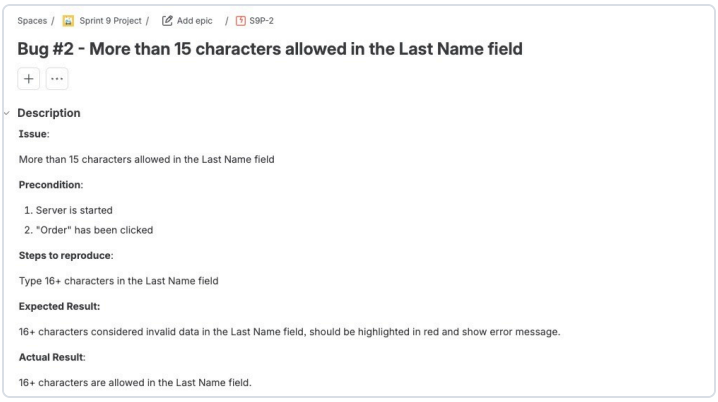


Figure 4. Jira report: issue, precondition, steps to reproduce, expected and actual result.

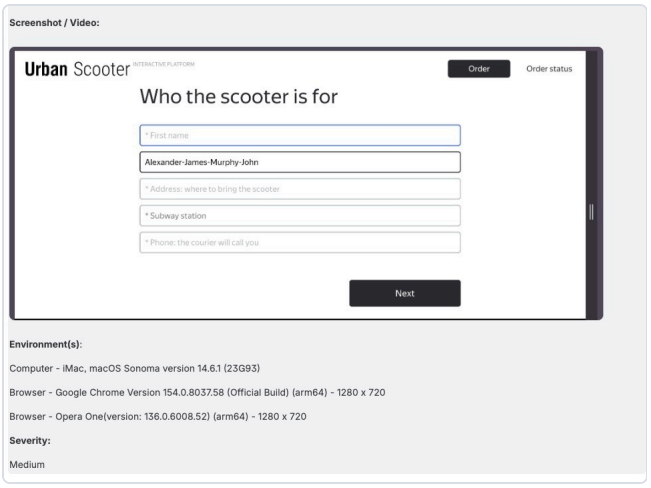


Figure 5. Same report: screenshot evidence, environment (both browsers) and severity.

Requirement	Last name: Latin letters, spaces and dashes; 2–15 characters. Invalid entries are highlighted in red with an error message.
Steps	Start the server, click “Order,” type 16 or more characters in the Last Name field.
Expected	16+ characters treated as invalid: field highlighted in red, error message shown.
Actual	16+ characters are accepted.
Boundary data	16 characters (Jonathon-Michael) and 17 characters (Jonathon-Michaele)
Traceability	Requirement → equivalence class (16, 17 characters) → test case 17 → Jira S9P-2 → reference link recorded in the workbook

Most important defects

1. **S9P-2, Last name accepts 16+ characters (Medium).** A stated limit is not enforced, so over-long data can enter the order.
2. **S9P-5, Address: no error shown (Medium).** Short or empty entries give no feedback, so a customer can proceed with an address that fails the rules.
3. **S9P-7 and S9P-8, Phone rules (Low–Medium).** A number without the required “+” is accepted, and the 10–12 length limit is misapplied. The form says the courier will call this number, so wrong rules can block valid numbers or admit unusable ones.

The other five bugs are Low: four are error-message wording and one is an Address length rule.

FULL-STACK MANUAL TESTING · PART 2 OF 3

Mobile Testing: Courier App

Urban Scooter Android app, tested in an emulator.

35	15 / 17 / 3	12	API 31
test cases designed and executed	passed / failed / skipped	distinct bugs logged	Pixel 5 emulator, Android 12

Scope

The assignment was to design and run test cases for the mobile features highlighted in the requirements, including their layout and design against Figma. Two features were in scope: **push notifications** for orders about to expire, and the **"No Internet connection" pop-up** that appears when a courier taps an active control without a connection. Results and Jira links went into a Google Sheets workbook.

Environment and preparation

Computer	iMac, macOS Sonoma 14.6.1 (23G93)
Emulator	Android Studio Quail 3 | 2026.1.3, Pixel 5 virtual device, Android 12 (API 31)
Tools	Android Studio emulator, Postman (setup data), Google Sheets (test cases), Jira (defects), Figma (design reference)
Setup	Installed the app APK on the emulator. Created a courier (POST /api/v1/courier) and an order (POST /api/v1/orders) in Postman. Entered the test server address through the "?" icon on the home screen, then logged in as the courier.
Techniques	Boundary-based test design; changing the emulator's date and time to reach time-based behavior without waiting; switching the network off and on; testing with the app active, minimized and with the screen off



Figure 1. Emulator home screen with the app installed.

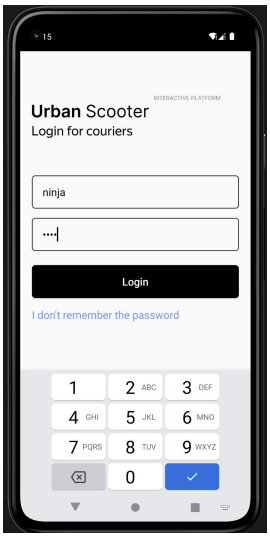


Figure 2. Courier login screen.

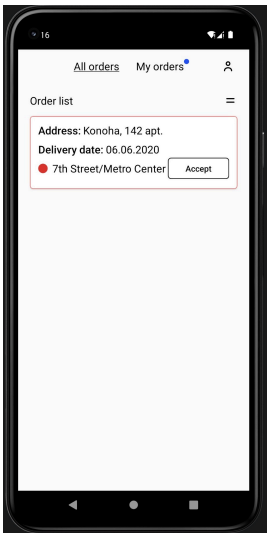


Figure 3. Order list after setup, with the test order.

Test design

I broke each highlighted requirement into checks, then wrote one case per check. Each case records its precondition, steps, expected result, actual result and status, with a Jira link when it failed.

AREA	REQUIREMENT UNDER TEST	CASES	WHAT THE CASES COVER
Push notification	A notification arrives when 2 hours remain to complete an order due by 11:59 p.m. (9:59 p.m.). It carries set text, and tapping it opens “My orders.”	12	Timing at the boundary: 9:58 p.m. (no notification), 9:59 p.m. (notification), 10:00 p.m. (no notification), each with the app active, minimized and the screen off (9 cases). Notification text, match to the Figma design, and tap-through to “My orders” (3 cases).
No Internet connection	With no connection, a “No Internet connection” pop-up appears when the user taps any active button on any screen. It closes only on “Ok” and returns if the connection is still down.	23	Offline taps on each control: “?” and URL entry, Login, “I don’t remember the password”, log-out with Yes and No, All orders, My orders, order filter, Apply, Accept, and Complete (19 cases). Pop-up dismissal and repetition (3 cases). Pop-up text and design against Figma (1 case).
Total		**35**	

> **How I reached the time-based cases.** The notification is due at 9:59 p.m. on the order’s date. Instead of waiting for that time, each case sets the emulator’s date to the order’s date and the time to 9:57 p.m., then waits for the target minute. This checks the minute before, at, and after the 2-hour mark.

	A	B	C	D	E	F	G
1	ID	Test Case	Precondition	Test Steps	Expected Result	Actual Result	Status
5	3	Verify the notification doesn't arrive more than 2 hours before order expiry while screen is off (9:58pm)	- Server is started - Courier is created - Order is created - Courier is logged in	1. Accept the order 2. Change phone date to the date of the order 3. Change phone time to 9:57pm 4. While screen is off, wait for 9:58pm	Notification doesn't appear.	Notification doesn't appear.	PASS
6	4	Verify the notification arrives 2 hours before order expiry while app is active (9:59pm)	- Server is started - Courier is created - Order is created - Courier is logged in	1. Accept the order 2. Change phone date to the date of the order 3. Change phone time to 9:57pm 4. While app is active, wait for 9:59pm	Notification appears.	Notification doesn't appear.	FAIL
7	5	Verify the notification arrives 2 hours before order expiry while app is minimized (9:59pm)	- Server is started - Courier is created - Order is created - Courier is logged in	1. Accept the order 2. Change phone date to the date of the order 3. Change phone time to 9:57pm 4. While app is minimized, wait for 9:59pm	Notification appears.	Notification doesn't appear.	FAIL
8	6	Verify the notification arrives 2 hours before order expiry while screen is off (9:59pm)	- Server is started - Courier is created - Order is created - Courier is logged in	1. Accept the order 2. Change phone date to the date of the order 3. Change phone time to 9:57pm 4. While screen is off, wait for 9:59pm	Notification appears.	Notification doesn't appear.	FAIL

Figure 4. Test case sheet excerpt, notification cases 3–6. Cases 4–6, at the 9:59 p.m. boundary, failed and link to a bug.

1	ID	Test Case	Precondition	Test Steps	Expected Result	Actual Result	Status
30	27	The "No Internet connection" pop-up is displayed when checking a box in the "Order location filter" without internet connection.	- Server is started - Courier is created - Order is created - Server address has been entered as the backend URL in the app - Courier is logged in	1. Turn off internet. 2. Click Order location filter 3. Select / checkbox a location	The "No Internet connection" pop-up is displayed	A location is selected, no pop-ups	FAIL
31	28	The "No Internet connection" pop-up is displayed when clicking "Apply" in the "Order location filter" without internet connection.	- Server is started - Courier is created - Order is created - Server address has been entered as the backend URL in the app - Courier is logged in	1. Turn off internet. 2. Click Order location filter 3. Click Apply	The "No Internet connection" pop-up is displayed	The "No Internet connection" pop-up is displayed	PASS
32	29	The "No Internet connection" pop-up is displayed when clicking "Accept" on an order without internet connection.	- Server is started - Courier is created - Order is created - Server address has been entered as the backend URL in the app - Courier is logged in	1. Turn off internet. 2. Click Accept on an order	The "No Internet connection" pop-up is displayed	The "No Internet connection" pop-up is displayed	PASS
33	30	The "No Internet connection" pop-up is displayed when clicking "Accept" on an order with internet access, and then "Yes" without internet access.	- Server is started - Courier is created - Order is created - Server address has been entered as the backend URL in the app - Courier is logged in	1. Click Accept on an order 2. Turn off internet. 2. Click "Yes"	The "No Internet connection" pop-up is displayed	The "No Internet connection" pop-up is displayed, but "Order accepted" also flashes.	FAIL

Figure 5. Test case sheet excerpt, “No Internet” cases 27–30. Each row records expected and actual results, status and a Jira link for failures.

Results

AREA	CASES	PASSED	FAILED	SKIPPED		BUGS
Push notification	12	6	3	3		1
No Internet connection	23	9	14	0		11
Total	**35**	**15**	**17**	**3**		**12**

Green = passed, red = failed, hatched = skipped. S9P numbers are reference IDs from the project's Jira board.

> **Skipped cases.** Three notification cases (text, match to Figma, tap-through) were skipped because no notification appeared to check. They depend on the S9P-10 fix and are not counted as passes or failures. **Failed cases are not the same as defects:** the 17 failures trace to 12 distinct bugs, since S9P-10 covers three cases and S9P-19 covers three.

Defects logged

JIRA	SCREEN	CASE	ACTION WHILE OFFLINE	ACTUAL RESULT	SEVERITY
Notification. Expected: a notification appears when 2 hours remain (9:59 p.m. for an order due at 11:59 p.m.).					
S9P-10	Notification	4, 5, 6	Wait for 9:59 p.m. with the app active, minimized, or the screen off	No notification appears	Medium–High
No internet connection (S9P-11 to S9P-21). Expected: with no connection, tapping any active button shows the "No Internet connection" pop-up, which stays until "Ok" is tapped.					
S9P-11	Login	16	Tap Login with incorrect details	"Invalid login" pop-up appears instead	Low
S9P-12	Login	18	Tap Login with correct details	"Unable to resolve host" flashes at the bottom	Low
S9P-13	Server setup	13	Tap the "?" icon	Backend URL field is revealed; no pop-up	Low
S9P-14	Server setup	14	Tap OK with an empty URL field	"Enter URL!" flashes; no pop-up	Low
S9P-15	Login	19	Tap "I don't remember the password"	"Contact manager" pop-up appears instead	Medium
S9P-16	Login	17	Tap OK on the login error	Returns to the home screen; no pop-up	Low
S9P-17	Orders	30	Tap Accept, go offline, then tap Yes	Offline pop-up appears, but "Order accepted" also flashes	Medium–High
S9P-18	Login	20	Tap OK on the "Contact manager" pop-up	Returns to the home screen; no pop-up	Low–Medium
S9P-19	Log-out	21, 22, 23	Tap Log-out, then Yes or No	"Log-out" pop-up appears; no offline pop-up on clicking Yes or No	Low–Medium
S9P-20	Order filter	26, 27	Open the order filter; select a location	Filter opens and a location is selected; no pop-up	Low–Medium
S9P-21	Pop-up	32	Tap outside "Ok" on the offline pop-up	Pop-up closes; it should stay until "Ok" is tapped	Low–Medium

Severity scale. Low: cosmetic, or a small departure from the written requirements. Medium: a potentially functional problem (such as a missing limit or feedback) with possible negative effects, but not catastrophic. High: a core feature fails, or a serious risk for this kind of app (such as security or data protection). Ranges mark borderline cases.

Featured defects

S9P-10: Two-hour notification does not appear

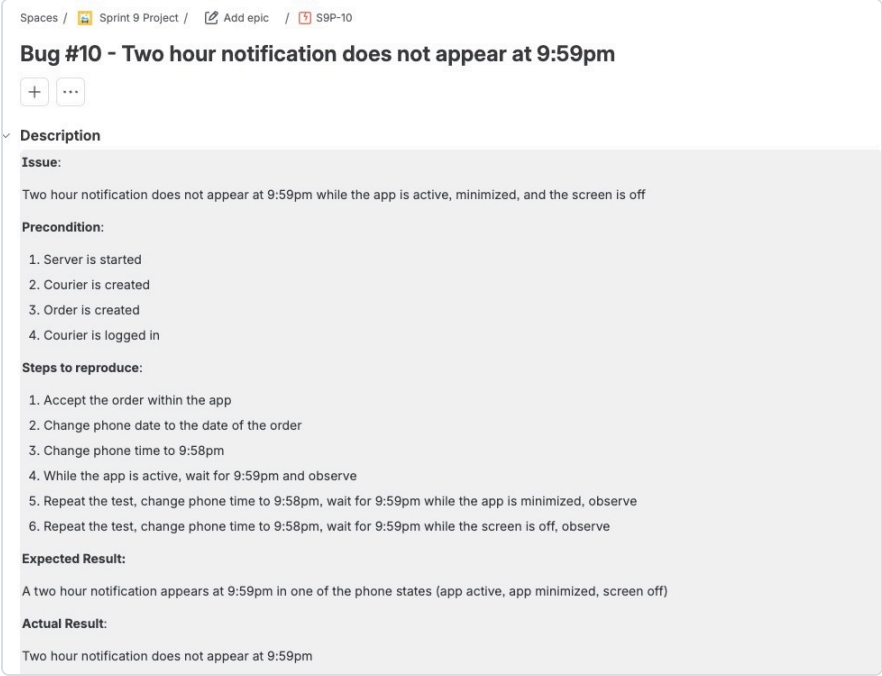


Figure 6. Jira report with preconditions, six steps, and expected and actual results.

Cases	4, 5, 6 (app active, minimized, screen off)
Expected	Notification at 9:59 p.m.
Actual	No notification in any of the three states
Severity	Medium–High
Effect	Blocked cases 10–12, which were skipped

S9P-17: “Order accepted” flashes offline

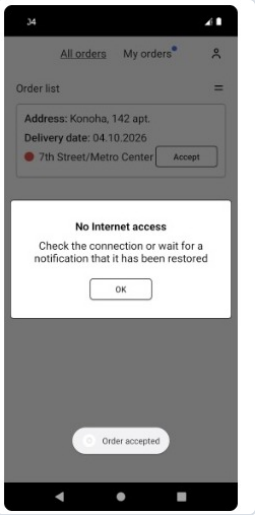


Figure 7. Offline pop-up shown correctly, with “Order accepted” also flashing at the bottom.

Case	30
Steps	Accept, go offline, tap Yes
Expected	Offline pop-up only
Actual	Pop-up plus a misleading “Order accepted”
Severity	Medium–High

Design check against Figma



Figure 8. Figma design of the pop-up (compare Figure 7).

Case 35 compared the pop-up with this Figma design, checking its text (“No Internet access. Check the connection or wait for a notification that it has been restored”) and layout. It passed. The notification’s own Figma comparison (case 11) was skipped because no notification appeared.

Most important defects

1. **S9P-10, no notification at 9:59 p.m. (Medium–High).** The courier gets no warning before an order deadline. It also blocked three other test cases.
2. **S9P-17, “Order accepted” flashes while offline (Medium–High).** The message could mislead the courier about whether the order was accepted.
3. **S9P-11 to S9P-21, missing or wrong feedback when offline.** Eleven controls respond differently from the required pop-up. Most are Low; S9P-15 is the highest at Medium.

FULL-STACK MANUAL TESTING · PART 3 OF 3

API Testing: Courier Add and Delete

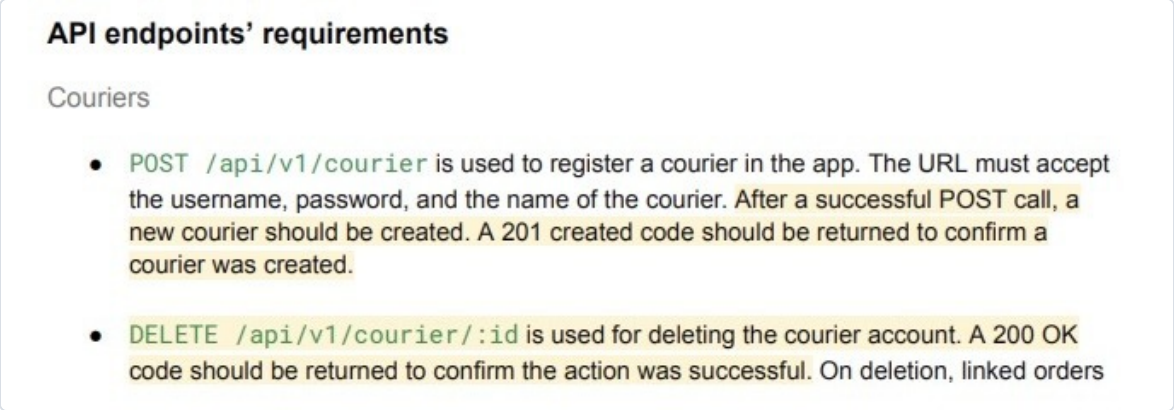
Urban Scooter back-end API, tested in Postman.

9	8 / 1	1	Postman
test cases designed and executed	passed / failed	distinct bug logged	JSON requests, status and body checks

Scope

The assignment was to study the back-end requirements and API documentation, design test cases for the highlighted **Add courier** and **Delete courier** requirements, run them, and file any bugs in Jira. Each case sends a request, then compares the status code and response body with the requirement and the API documentation.

ENDPOINT	REQUIREMENT UNDER TEST	SUCCESS CODE
POST /api/v1/courier	Registers a courier from a login, password and first name.	201 Created
DELETE /api/v1/courier/:id	Deletes the courier account with the given ID.	200 OK



API endpoints' requirements

Couriers

- `POST /api/v1/courier` is used to register a courier in the app. The URL must accept the username, password, and the name of the courier. After a successful POST call, a new courier should be created. A 201 created code should be returned to confirm a courier was created.
- `DELETE /api/v1/courier/:id` is used for deleting the courier account. A 200 OK code should be returned to confirm the action was successful. On deletion, linked orders

Figure 1. Back-end requirements for the courier endpoints, with the tested requirements highlighted.

Environment and tools

Computer	iMac, macOS Sonoma 14.6.1 (23G93)
Tools	Postman (requests and responses), Google Sheets (test cases), Jira (defect), API documentation served at the test server's /docs/en/ page
Method	Positive, negative and missing-input cases for each endpoint; expected status code and response body written before each run

Test cases and results

I wrote cases for the success response of each endpoint, then for the error responses the documentation describes: a duplicate login, missing required fields, a missing ID, and an ID that does not exist. Test data in the workbook uses placeholder login, password and name values.

ID	ENDPOINT	WHAT IS TESTED	EXPECTED	ACTUAL	RESULT
1	Create	Valid data for all parameters	201 Created, {ok: true}	Same	Pass
2	Create	Duplicate login	409 Conflict, “This login is not available. Try another.”	Same	Pass
3	Create	Login and password, no first name	201 Created, {ok: true}	Same	Pass
4	Create	Password and first name, no login	400 Bad Request, “Not enough data to create an account.”	Same	Pass
5	Create	Login and first name, no password	400 Bad Request, “Not enough data to create an account.”	Same	Pass
6	Delete	Existing courier ID	200 OK, {ok: true}	Same	Pass
7	Delete	Courier that was already deleted	404 Not Found, “There’s no courier with this ID.”	Same	Pass
8	Delete	No ID in the request	400 Bad Request, “Not enough data to remove the courier”	404 Not Found, “Not Found.”	Fail **S9P-22**
9	Delete	ID that does not exist	404 Not Found, “There’s no courier with this ID.”	Same	Pass

“Same” means the actual response matched the expected one. Create = POST /api/v1/courier; Delete = DELETE /api/v1/courier/:id.

Results

ENDPOINT	CASES	PASSED	FAILED	BUGS
Create courier	5	5	0	0
Delete courier	4	3	1	1
Total	**9**	**8**	**1**	**1**



Green = passed, red = failed.

Defect logged

JIRA	REQUEST	CASE	EXPECTED	ACTUAL	SEVERITY
S9P-22	DELETE /api/v1/courier/ with an empty ID	8	400 Bad Request, “Not enough data to remove the courier”	404 Not Found, “Not Found.”	Low–Medium

Severity scale. Low: cosmetic, or a small departure from the written requirements. Medium: a potentially functional problem (such as a missing limit or feedback) with possible negative effects, but not catastrophic. High: a core feature fails, or a serious risk for this kind of app (such as security or data protection). Ranges mark borderline cases.

Featured defect: S9P-22

Deleting a courier without an ID returns 404 instead of 400. Case 8. Low–Medium severity.

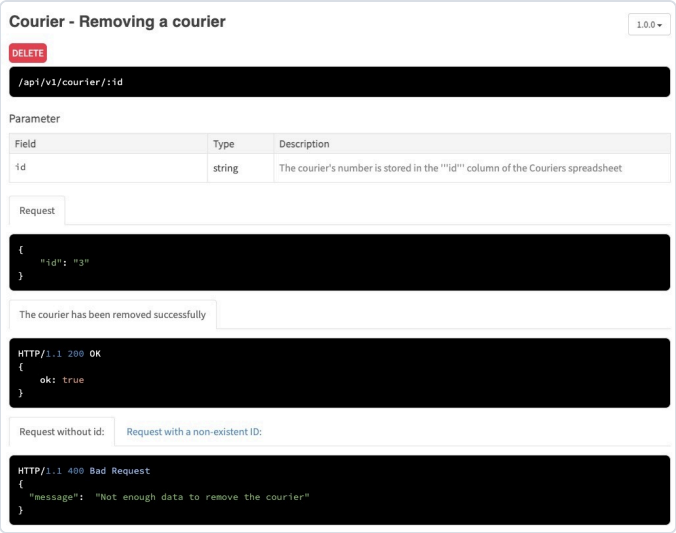


Figure 2. Expected, per the API documentation: a request without an ID returns 400 Bad Request with “Not enough data to remove the courier.”

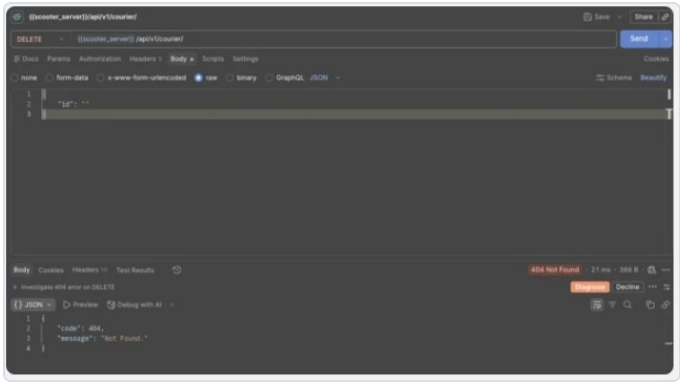


Figure 3. Actual, in Postman: DELETE /api/v1/courier/ with an empty ID returns 404 Not Found with “Not Found.”

Requirement	A DELETE request without a courier ID should be rejected as incomplete.
Steps	Start the server, then send a DELETE request to /api/v1/courier/ with an empty ID.
Expected	HTTP 400 Bad Request, message “Not enough data to remove the courier”
Actual	HTTP 404 Not Found, code 404, message “Not Found.”
Traceability	Requirement → test case 8 → Jira S9P-22 → reference link recorded in the workbook

Most important defects

1. **S9P-22, wrong status code for a missing ID (Low–Medium).** The API answers an incomplete request as “not found,” so a client cannot distinguish between a malformed call and a missing courier, and the expected message is not returned. It was the only failure among nine cases.

SUMMARY

Results and Automation Potential

Summary of results

AREA	CASES	PASSED	FAILED	SKIPPED	BUGS	WHERE THE BUGS WERE
Web	65	54	11	0	9	Input validation rules and error-message wording
Mobile	35	15	17	3	12	Missing or wrong feedback when offline; missing notification
API	9	8	1	0	1	Wrong status code for a request without an ID
Total	**109**	**77**	**29**	**3**	**22**	

Automation potential

All of these cases were run manually. Because they are repeatable, they could be automated and re-run after every change to catch regressions.

- **Web:** the 60 field-validation cases follow one pattern: enter a value, check whether the field accepts it or shows the error. A script could run them from a table of inputs and expected results, in both browsers (for example with Playwright).
- **Mobile:** the 23 offline cases mostly follow one pattern: go offline, tap a control, check for the pop-up. A script could run them on the emulator, switching the network off and on and setting the clock for the notification cases (for example with Appium).
- **API:** the 9 cases are requests with expected status codes and messages, so they could run as a Postman collection with tests, or as a script.